

CSE 200 — TECHNICAL WRITING AND PRESENTATION
BANGLADESH UNIVERSITY OF ENGINEERING AND
TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Inside BERT

Architecture, Pre-training, and Fine-tuning

March 2026

Author	Your Name
Student ID	XXXXXXX
Subsection	A2
Group	—

Abstract. Bidirectional Encoder Representations from Transformers (BERT) represents a paradigm shift in Natural Language Processing (NLP). By leveraging deep bidirectional context within a Transformer encoder stack, BERT achieves state-of-the-art performance across a wide spectrum of language understanding tasks. This report presents a comprehensive study of BERT, encompassing its architectural foundations, self-attention mechanisms, input representations, pre-training objectives, and fine-tuning strategies. Theoretical insights are grounded in rigorous mathematical formulations, and practical implications are discussed in the context of downstream task adaptation. The report further analyses BERT's contributions relative to prior sequential models and outlines directions for future research in the field.

Contents

1	Introduction	7
2	Motivation: The Limits of Static Word Embeddings	7
2.1	What Are Word Embeddings?	7
2.2	The Fatal Flaw: Context-Blindness	8
3	Sequential Models: LSTM and BiLSTM	8
3.1	Recurrent Neural Networks	8
3.2	Long Short-Term Memory (LSTM)	8
3.3	Limitations of the LSTM	9
3.4	Bidirectional LSTM (BiLSTM)	9
4	The Transformer Architecture	10
4.1	Motivation and Core Idea	10
4.2	Self-Attention Mechanism	10
4.3	Multi-Head Attention	11
4.4	Positional Encoding	11
4.5	Encoder and Decoder Stacks	11
4.6	Why the Transformer Surpasses the LSTM	12
5	Introduction to bert	13
5.1	From Transformer to BERT	13
5.2	What Makes BERT Truly Bidirectional?	14
6	Why bert Was a Turning Point	14
6.1	Pre-training Objectives	14
6.1.1	Masked Language Model (MLM)	14
6.1.2	Next Sentence Prediction (NSP)	15
6.2	Fine-tuning Paradigm	15
6.3	Impact: State-of-the-Art Results	15
6.4	Why It Was a Paradigm Shift	15

7	High-Level Architecture Overview	16
7.1	Input Representation	16
7.2	Encoder Stack	16
8	BERT Architecture	16
8.1	Transformer Encoder Stack	17
8.2	Hierarchical Representation Learning	18
8.3	Model Configurations	19
9	Self-Attention Mechanism	19
9.1	Conceptual Overview	19
9.2	Mathematical Formulation	19
9.3	Interpretation of Query, Key, and Value Roles	20
9.4	Multi-Head Attention	21
9.5	Position-Wise Feed-Forward Network	21
10	Input Representation	21
10.1	Tokenisation	21
10.2	Embedding Components	22
10.3	Special Tokens	22
11	Pre-training Strategy	23
11.1	Masked Language Modelling (MLM)	23
11.2	Next Sentence Prediction (NSP)	24
11.3	Pre-training Corpora	24
11.4	Key Observations	24
12	Fine-Tuning	25
12.1	General Procedure	25
12.2	Task Formulations	25
12.2.1	Single-Sentence Classification	25
12.2.2	Sentence-Pair Classification	25
12.2.3	Question Answering (Extractive)	26
12.2.4	Named Entity Recognition (NER)	26
12.3	Practical Considerations	27
13	Discussion and Impact	28
13.1	Key Contributions	28
13.2	Advantages	28
13.3	Limitations	28
13.4	Influence and Subsequent Work	29
13.5	Conclusion	29
14	Applications of BERT	29
14.1	Sentiment Analysis	30
14.2	Search Intent Understanding — Google Search Integration	30
14.3	Question Answering	31
14.4	Named Entity Recognition	32
15	Limitations of BERT	33

16 What Came After BERT	34
16.1 Training and Efficiency Improvements	34
16.2 Context and Modelling Extensions	35
16.3 Deployment-Friendly Models	35
17 Conclusion	35
References	37

List of Figures

1	LSTM encoder–decoder for sequence-to-sequence translation (“ <i>I am happy</i> ” $\rightarrow$ “ <i>Je suis . . .</i> ”). The encoder processes tokens sequentially, passing hidden state h_t forward; the decoder generates the target sequence conditioned on the final encoder state.	9
2	Bidirectional LSTM (BiLSTM). A forward pass (teal, A) processes tokens left-to-right; a backward pass (cyan, A') processes tokens right-to-left. The outputs are concatenated at each position to form a context-aware representation. The two passes are trained independently and merged only at the output.	10
3	Self-attention in action. The pronoun “ <i>it</i> ” attends most strongly to “ <i>animal</i> ”, resolving the coreference in a single layer without traversing a chain of timesteps. Thicker arcs indicate higher attention weights.	11
4	The original Transformer architecture [9]. The encoder (left) produces contextual embeddings from the source sequence. The decoder (right) generates the target sequence auto-regressively, using masked self-attention and cross-attention over the encoder output.	12
5	Architectural comparison. The original Transformer uses both encoder and decoder for translation. BERT retains only the encoder stack, using the [CLS] token representation as input to task-specific classifier heads.	13
6	Stacked Transformer encoder architecture underlying BERT. Each of the L encoder layers applies Multi-Head Attention followed by a Feed-Forward Network, with residual connections and layer normalisation at each sub-layer.	18
7	Attention mechanisms in BERT. (Left) Scaled dot-product attention computes relevance scores from queries and keys, applies softmax normalization to obtain attention weights $\mathbf{A}$, then aggregates values. (Right) Multi-head attention applies h parallel scaled dot-product functions in projected subspaces.	20
8	BERT input representation for a sentence pair. Each token embedding is summed with the corresponding segment embedding and position embedding. The special [CLS] token is prepended and [SEP] tokens delimit sentences.	22
9	BERT fine-tuning architectures for four task types. (a) Sentence-pair classification; (b) single-sentence classification; (c) extractive question answering; (d) sequence labelling (NER). Only the top-layer head changes across tasks; the encoder is shared.	27
10	Impact of BERT on Search Intent Resolution. The left panel (Without BERT) shows results focusing on U.S. citizens traveling <i>to</i> Brazil, failing to capture the directional intent. The right panel (With BERT) correctly identifies the query “2019 brazil traveler to usa” as a request for U.S. entry requirements for Brazilian nationals.	30
11	Close-up of BERT Contextual Understanding. Detailed view of the third-party intent resolution for pharmacy pickup queries.	31

12	Four-Stage Named Entity Recognition (NER) Pipeline. The diagram illustrates the transition from (1) raw input strings to (2) contextual candidate detection via BERT, followed by (3) token-level classification into categories such as Person , GPE , and Location , and finally (4) the consolidation of labels into a structured output table.	32
13	Limitations of BERT. Fixed context window, encoder-only design, high computational cost, weak reasoning ability, inherited data bias, and dependence on task-specific fine-tuning.	33
14	BERT Successors and Improvements. Overview of key BERT-based models that address limitations in pre-training, model efficiency, context length, reasoning, and task adaptability.	34

List of Tables

1	Summary of static word embedding models.	7
2	LSTM vs. Transformer: a direct comparison.	13
3	Original Transformer vs. BERT.	14
4	Selected benchmark results at the time of BERT's release [10].	15
5	BERT model variants [10].	16
6	Computation flow within a single Transformer encoder layer. MHSA denotes Multi-Head Self-Attention; FFN denotes Position-Wise Feed-Forward Network.	17
7	Standard BERT model configurations. L : number of encoder layers; H : hidden dimension; A : number of attention heads; P : total parameter count.	19
8	Attention head configuration for standard BERT models. Each head operates independently in a projected subspace before concatenation.	21
9	WordPiece tokenisation examples. Rare and out-of-vocabulary words are decomposed into frequent subword units.	22
10	Masked Language Modelling (MLM) token treatment strategy. For each of the 15% selected tokens, the replacement is chosen probabilistically.	24
11	BERT fine-tuning results on representative NLU benchmarks. GLUE is a composite score; others report default metrics (Acc = accuracy, F1 = harmonic mean).	26
12	BERT and contemporary models: a comparison of key properties. All models use the Transformer encoder architecture; differences lie in pre-training objectives, corpus size, and downstream performance.	29
13	The evolution of NLP capabilities introduced by BERT.	36

1. Introduction

Understanding language is one of the hardest challenges in artificial intelligence. Even humans rely on enormous amounts of world knowledge and contextual reasoning to interpret a single sentence. For machines, the challenge begins at the most basic unit of language: the word.

A central difficulty is *polysemy* — the fact that most words carry multiple possible meanings, and the correct meaning is determined entirely by context. Consider the sentence:

*“I went to the **bank** to deposit money, then sat by the river **bank**.”*

The word *bank* appears twice, yet refers to entirely different real-world entities. Any NLP system — a search engine, a chatbot, or a machine translator — must resolve this ambiguity correctly to produce a useful output.

This report traces the progression of ideas that led researchers to BERT, examining why each intermediate approach fell short and what design choices in BERT finally addressed the core problem. Section 2 covers static word embeddings. Section 3 covers LSTMs and BiLSTMs. Section 4 introduces the Transformer. Sections 5 and 6 present BERT itself and its significance. Section 7 provides a high-level architecture overview.

2. Motivation: The Limits of Static Word Embeddings

2.1 What Are Word Embeddings?

The first major breakthrough beyond symbolic NLP was the introduction of *distributed word representations*, or *word embeddings*: mappings from vocabulary items to dense vectors in a continuous d -dimensional space $\mathbb{R}^d$.

The foundational idea is the **distributional hypothesis** [1]: words that occur in similar contexts tend to carry similar meanings. Models such as Word2Vec [2], GloVe [3], and FastText [4] operationalise this by training on large text corpora to produce an embedding matrix $\mathbf{E} \in \mathbb{R}^{|V| \times d}$, where each row $\mathbf{e}_w$ is the embedding of word w .

Word2Vec [2] (Google, 2013) trains a shallow neural network with either a Skip-gram or CBOW (Continuous Bag-of-Words) objective to predict a word from its context or vice versa. **GloVe** [3] (Stanford, 2014) learns embeddings from the global word co-occurrence matrix. **FastText** [4] (Facebook, 2016) extends Word2Vec to subword units, improving handling of morphologically rich languages and out-of-vocabulary words.

Table 1: Summary of static word embedding models.

Model	Origin	Year	Key Innovation
Word2Vec [2]	Google	2013	Skip-gram / CBOW objectives; scalable training
GloVe [3]	Stanford	2014	Global co-occurrence matrix factorisation
FastText [4]	Facebook	2016	Subword n -gram representations

These models yield rich geometric properties. The famous analogy $\vec{\text{king}} - \vec{\text{mān}} + \vec{\text{wo}}\vec{\text{man}} \approx \vec{\text{quēen}}$ emerges naturally from the learned space, illustrating that semantic and syntactic relationships are captured as linear offsets in the embedding space.

2.2 The Fatal Flaw: Context-Blindness

Despite their utility, static embeddings have one fundamental limitation: **every occurrence of a word receives the same vector**, regardless of the sentence it appears in. Formally, the mapping $f: w \mapsto \mathbf{e}_w$ is a simple lookup — context plays no role.

For the polysemy example above, both occurrences of *bank* map to an identical vector $\mathbf{e}_{\text{bank}} \in \mathbb{R}^d$. The embedding must somehow encode both the financial-institution sense and the geographical sense simultaneously, which it cannot do faithfully. This context-blindness makes static embeddings inadequate for tasks that require genuine language understanding.

3. Sequential Models: LSTM and BiLSTM

3.1 Recurrent Neural Networks

To incorporate context into word representations, researchers turned to **Recurrent Neural Networks (RNNs)**, which process a sequence of tokens $x_1, x_2, \dots, x_T$ one step at a time, maintaining a hidden state h_t that accumulates information from previous tokens:

$$h_t = f_\theta(h_{t-1}, x_t) \quad (1)$$

where f_θ is a learned function. In principle, h_T encodes the entire history of the sequence. In practice, simple RNNs suffer from *vanishing gradients* [5]: gradients shrink exponentially as they are back-propagated through many timesteps, making it impossible to learn long-range dependencies.

3.2 Long Short-Term Memory (LSTM)

The **Long Short-Term Memory (LSTM)** [6] (Hochreiter & Schmidhuber, 1997) addresses the vanishing gradient problem by introducing a *cell state* c_t alongside the hidden state, regulated by three learned gates: the *forget gate* f_t , the *input gate* i_t , and the *output gate* o_t . The full update equations are:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f) \quad (2)$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \quad (3)$$

$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c) \quad (4)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (5)$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \quad (6)$$

$$h_t = o_t \odot \tanh(c_t) \quad (7)$$

where σ is the sigmoid function, $\odot$ denotes element-wise multiplication, and $[h_{t-1}, x_t]$ denotes concatenation.

The cell state c_t (Eq. 5) acts as a *memory conveyor belt*: the forget gate decides what to discard from the previous state, and the input gate controls what new information to write. This mechanism allows gradients to flow across hundreds of timesteps, resolving the vanishing gradient problem that plagued simple RNNs.

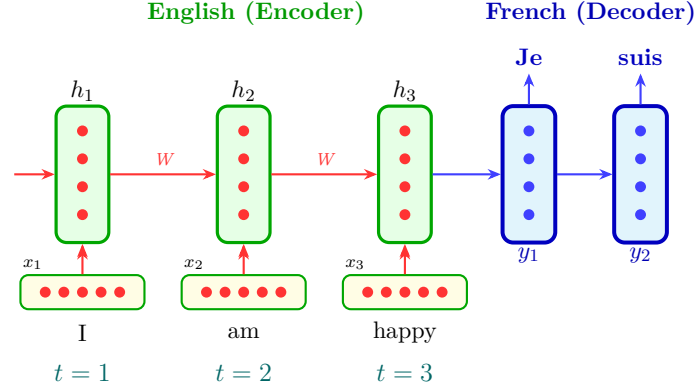


Figure 1: LSTM encoder–decoder for sequence-to-sequence translation (“*I am happy*” → “*Je suis ...*”). The encoder processes tokens sequentially, passing hidden state h_t forward; the decoder generates the target sequence conditioned on the final encoder state.

3.3 Limitations of the LSTM

While the LSTM addresses the vanishing gradient problem, it has three remaining weaknesses that limit its effectiveness for language understanding:

1. Sequential bottleneck.

The recurrence $h_t = f(h_{t-1}, x_t)$ is inherently sequential: token t cannot be processed until token $t-1$ is complete. This prevents parallelisation along the time axis, making training slow on long sequences.

2. Memory degradation.

Despite the gating mechanism, information from early tokens is progressively diluted over many timesteps. For a sequence of length T , the influence of x_1 on h_T decays roughly as $\prod_{t=2}^T f_t$ — a product of forget gate values all ≤ 1 .

3. Unidirectionality.

A left-to-right LSTM builds the representation of token t from only its left context $x_1, \dots, x_{t-1}$. The right context — which is often crucial for disambiguation — is unavailable.

3.4 Bidirectional LSTM (BiLSTM)

The **Bidirectional LSTM** [7] (BiLSTM) attempts to address the unidirectionality problem by running two separate LSTM passes over the input — a forward pass (left-to-right) producing $\vec{h}_t$ and a backward pass (right-to-left) producing $\overleftarrow{h}_t$ — and concatenating their outputs:

$$h_t = [\vec{h}_t; \overleftarrow{h}_t] \quad (8)$$

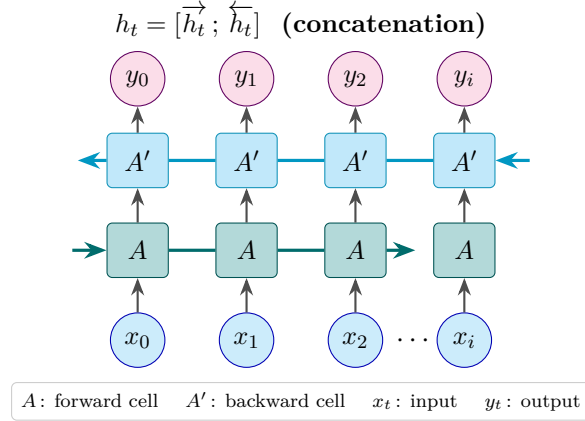


Figure 2: Bidirectional LSTM (BiLSTM). A forward pass (teal, A) processes tokens left-to-right; a backward pass (cyan, A') processes tokens right-to-left. The outputs are concatenated at each position to form a context-aware representation. The two passes are trained independently and merged only at the output.

The BiLSTM provides richer representations than a unidirectional LSTM. ELMo [8] (Peters et al., 2018), a prominent pre-BERT contextualised embedding model, uses a stacked BiLSTM and demonstrated that contextualised representations transfer effectively to downstream tasks. However, the BiLSTM still inherits two critical limitations:

- **Not truly bidirectional.** The forward and backward passes are trained *separately* and only combined at the output (Eq. 8). The two directions never interact during feature extraction; the model cannot, within a single layer, allow left-context and right-context information to jointly influence each other.
- **Still sequential.** Both passes remain sequential along their respective directions, limiting parallelisation and scalability.

4. The Transformer Architecture

4.1 Motivation and Core Idea

Both LSTM variants process text token by token. The root cause of their limitations is the sequential recurrence: dependency on the previous hidden state forces one-at-a-time computation and creates an information bottleneck for long-range dependencies.

The **Transformer** [9] (Vaswani et al., 2017) discards recurrence entirely. Its central mechanism is **self-attention**: every token directly attends to every other token in the sequence *simultaneously*, computing a weighted combination of all position representations in a single operation.

4.2 Self-Attention Mechanism

Given an input sequence of n tokens represented as a matrix $X \in \mathbb{R}^{n \times d_{\text{model}}}$, three linear projections produce query, key, and value matrices:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V \quad (9)$$

where $W^Q, W^K, W^V \in \mathbb{R}^{d_{\text{model}} \times d_k}$. The **scaled dot-product attention** output is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \quad (10)$$

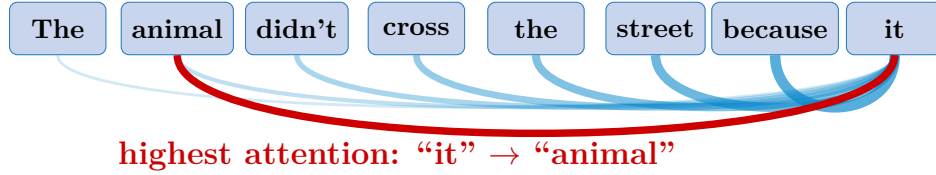
The scaling factor $1/\sqrt{d_k}$ prevents the dot products from growing large in magnitude, which would push the softmax into regions of extremely small gradients. Each output row is a weighted sum of all value vectors, where the weights reflect how relevant each token is to the current query position.

4.3 Multi-Head Attention

Rather than performing a single attention operation, the Transformer applies h *parallel* attention heads, each learning a different projection:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O \quad (11)$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$ and $W^O \in \mathbb{R}^{hd_k \times d_{\text{model}}}$. Multiple heads allow the model to capture different types of relationships simultaneously — *e.g.* syntactic agreement, coreference, and semantic similarity — from the same input.



Each token attends to *all* others simultaneously (in parallel). Long-range links resolved in one step.

Figure 3: Self-attention in action. The pronoun “it” attends most strongly to “animal”, resolving the coreference in a single layer without traversing a chain of timesteps. Thicker arcs indicate higher attention weights.

4.4 Positional Encoding

Since self-attention is permutation-invariant (Eq. 17 has no notion of token order), the Transformer injects **positional encodings** [9] into the input embeddings:

$$\text{PE}_{(\text{pos}, 2i)} = \sin\left(\frac{\text{pos}}{10000^{2i/d}}\right), \quad \text{PE}_{(\text{pos}, 2i+1)} = \cos\left(\frac{\text{pos}}{10000^{2i/d}}\right) \quad (12)$$

These fixed sinusoidal functions encode absolute position and allow the model to represent relative distances between tokens.

4.5 Encoder and Decoder Stacks

The original Transformer [9] is an encoder–decoder model. Each encoder layer consists of (i) multi-head self-attention and (ii) a position-wise feed-forward network (FFN), each sublayer wrapped with a residual connection and Layer Normalisation. The decoder additionally contains a *cross-attention* sublayer that queries the encoder output.

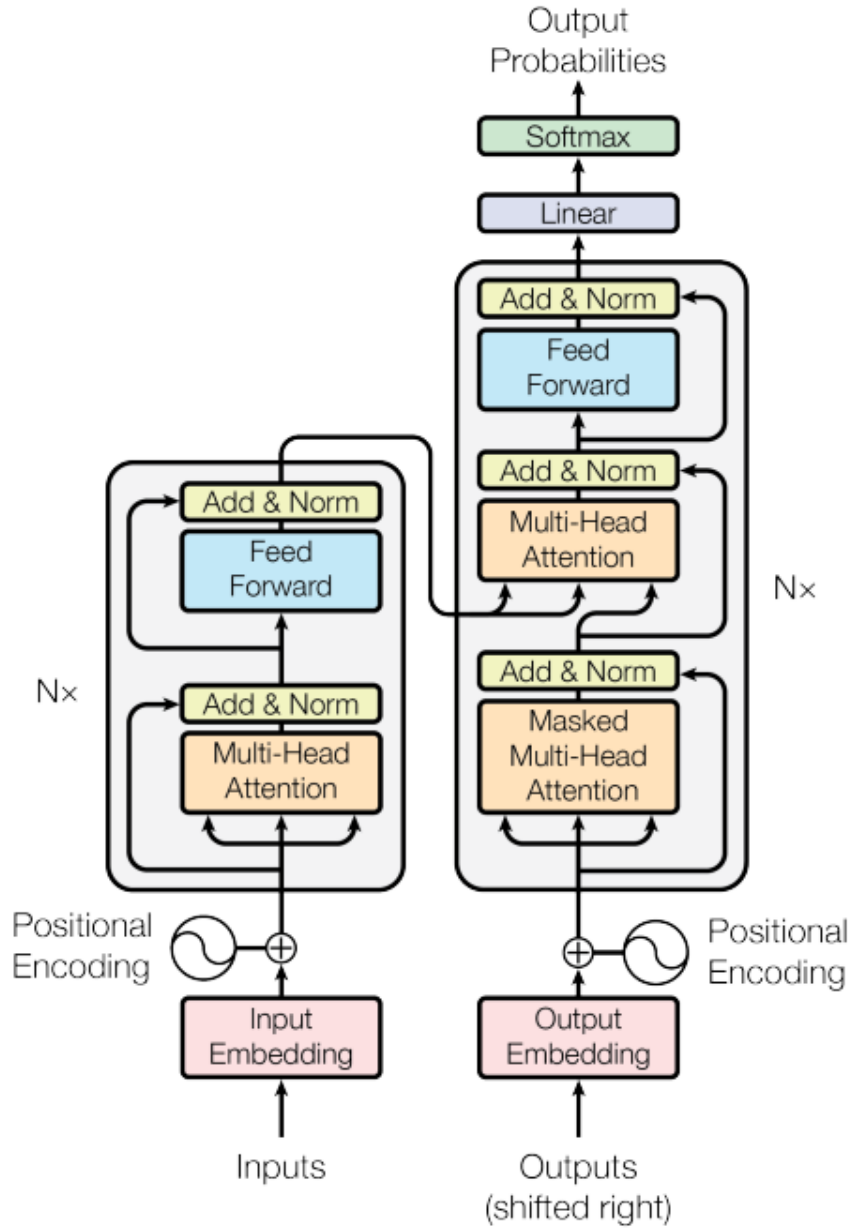


Figure 4: The original Transformer architecture [9]. The encoder (left) produces contextual embeddings from the source sequence. The decoder (right) generates the target sequence auto-regressively, using masked self-attention and cross-attention over the encoder output.

4.6 Why the Transformer Surpasses the LSTM

Table 2 summarises the key differences between the two architectures.

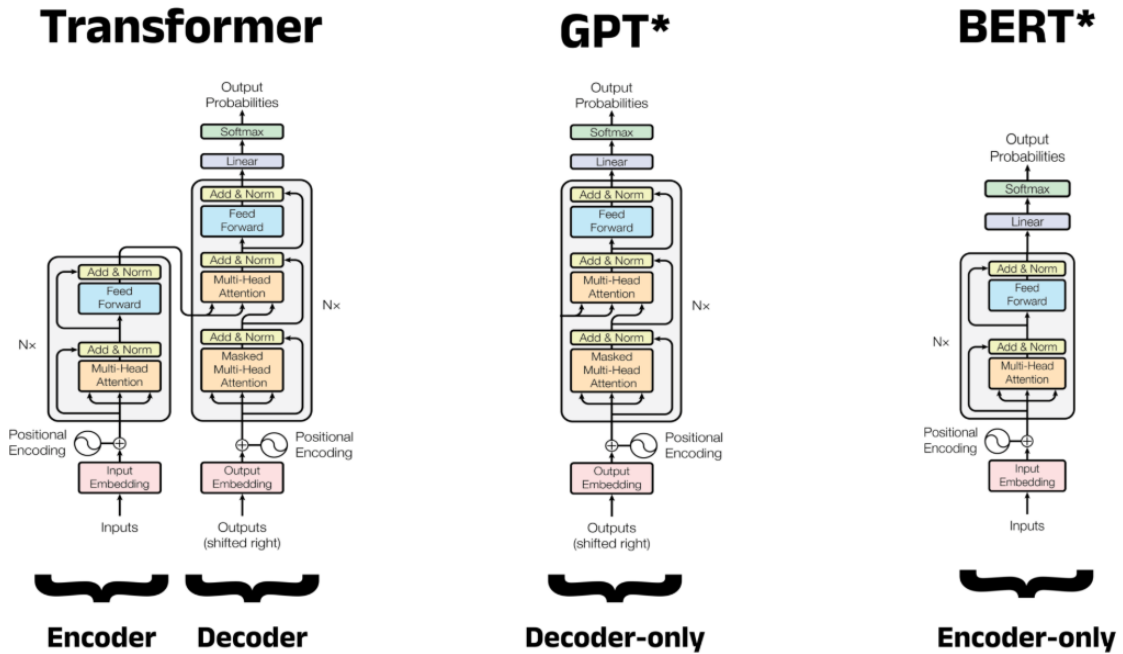
Table 2: LSTM vs. Transformer: a direct comparison.

Property	LSTM / BiLSTM	Transformer
Parallelism	Sequential; one step per token	Fully parallel over all tokens
Long-range dependencies	Degrade with distance	Constant path length between any two positions
Bidirectionality	Separate forward/backward passes	Joint left+right context within every attention layer
Scalability	Hard to scale to very long sequences	Scales with hardware via parallelism

5. Introduction to bert

5.1 From Transformer to bert

The Transformer’s encoder already attends bidirectionally: in each self-attention head, every token can query every other token regardless of direction. However, the original Transformer was designed as an encoder–decoder for sequence-to-sequence tasks. **bert** [10] (Devlin et al., 2018) makes a deliberate choice: *keep only the encoder stack* and pre-train it for language understanding rather than generation.



*Illustrative example, exact model architecture may vary slightly

Figure 5: Architectural comparison. The original Transformer uses both encoder and decoder for translation. BERT retains only the encoder stack, using the [CLS] token representation as input to task-specific classifier heads.

Table 3 contrasts the two architectures.

Table 3: Original Transformer vs. BERT.

Aspect	Original Transformer [9]	bert [10]
Architecture	Encoder + Decoder	Encoder only (N stacked layers)
Primary task	Sequence-to-sequence (translation)	Representation learning / understanding
Context	Decoder is left-to-right (masked)	Fully bidirectional at every layer
Pre-training	None (direct supervision)	Masked LM + Next Sentence Prediction
Usage	Train end-to-end per task	Pre-train once, fine-tune per task

Remark 5.1. “The Transformer learns to *translate*. BERT learns to *understand*.”

5.2 What Makes bert Truly Bidirectional?

This is a subtle but crucial point. ELMo [8] was also called “bidirectional,” but its forward and backward LSTMs were trained with separate language model objectives and then concatenated. BERT, by contrast, uses a *single unified* Transformer encoder in which every attention head simultaneously sees all tokens from both directions, and every layer integrates left and right context jointly. This *deep* bidirectionality — as opposed to shallow concatenation — is what makes BERT’s representations qualitatively different.

6. Why bert Was a Turning Point

6.1 Pre-training Objectives

BERT is pre-trained on two self-supervised tasks using only unlabelled text (Wikipedia + BookCorpus, ≈ 3.3 billion words):

6.1.1 Masked Language Model (MLM)

Inspired by the Cloze task [11], 15% of input tokens are randomly masked. BERT is trained to predict the original token from its *bidirectional* context:

$$\mathcal{L}_{\text{MLM}} = - \sum_{i \in \mathcal{M}} \log P(x_i \mid x_1, \dots, x_{i-1}, [\text{MASK}], x_{i+1}, \dots, x_n) \quad (13)$$

where $\mathcal{M}$ is the set of masked positions. Because a correct prediction requires integrating context from *both* sides, this objective inherently forces the model to learn deeply bidirectional representations. Traditional left-to-right language models cannot use this objective without a fundamental architectural change.

6.1.2 Next Sentence Prediction (NSP)

Many NLP tasks (*e.g.* question answering and natural language inference) require understanding the *relationship* between two sentences. BERT is therefore also trained on a binary classification task: given a sentence pair (A, B) , predict whether B is the sentence that actually follows A in the corpus or a randomly sampled sentence. This trains the model to encode inter-sentence coherence in the [CLS] token.

6.2 Fine-tuning Paradigm

After pre-training, BERT’s weights serve as a starting point for any downstream NLP task. A lightweight task-specific head is appended:

- **Classification** (sentiment, topic): linear layer on the [CLS] representation.
- **Token labelling / NER**: token-level classifier on each output position.
- **Extractive QA**: two linear layers predicting span start and end positions.
- **Natural Language Inference**: classification on [CLS] for entailment / contradiction / neutral.

Fine-tuning requires only 2–4 epochs, making BERT highly data-efficient. The same pre-trained model handles all four task types with no structural changes to the core encoder.

6.3 Impact: State-of-the-Art Results

Upon release, BERT set new state-of-the-art results across eleven NLP benchmarks [10]:

Table 4: Selected benchmark results at the time of BERT’s release [10].

Benchmark	Task	Prior SOTA	bert
GLUE [12]	Multi-task NLU	72.8	80.4
SQuAD v1.1 [13]	Reading comprehension (F1)	91.7	93.2
SQuAD v2.0 [14]	Unanswerable QA (F1)	78.0	83.1
MultiNLI [15]	Natural language inference	86.7	86.7 ⁺

On SQuAD v1.1, BERT exceeded human performance on the F1 metric. Crucially, these results came from a *single* pre-trained model fine-tuned separately for each task, demonstrating that rich, general-purpose language representations can be learned once and deployed everywhere.

6.4 Why It Was a Paradigm Shift

Four reasons make BERT a genuine turning point in NLP:

1. **Deep bidirectionality at scale.** Unlike ELMo’s shallow concatenation of separately trained LSTMs, BERT integrates left and right context jointly in every layer, yielding fundamentally richer token representations.
2. **Transfer learning for NLP.** BERT demonstrated that the “pre-train on unlabelled data, fine-tune on small labelled data” paradigm — long established in computer vision — works powerfully for language. This shifted NLP research away from task-specific architectures towards general-purpose pre-trained models.

3. **Open release.** Google released BERT’s weights, code, and pre-training procedure publicly, enabling rapid community adoption and spawning a generation of derivative models: RoBERTa [16], DistilBERT [17], ALBERT [18], domain-specific variants, and multilingual extensions.
4. **Contextual representations by design.** Every output embedding depends on the full sentence context through every attention layer, resolving polysemy as an architectural consequence rather than an optional post-processing step.

7. High-Level Architecture Overview

This section briefly describes BERT’s architecture. Detailed analysis of each component will be covered in the subsequent team sections.

7.1 Input Representation

BERT constructs each input token’s embedding as the sum of three learned components [10]:

$$E_{\text{input}} = E_{\text{token}} + E_{\text{segment}} + E_{\text{position}} \quad (14)$$

where E_{token} is a WordPiece [19] subword embedding ($|V| \approx 30,000$), E_{segment} distinguishes sentence A from sentence B in a paired input, and E_{position} is a learned positional embedding. A special [CLS] token is prepended to every input; [SEP] marks sentence boundaries.

7.2 Encoder Stack

BERT stacks N identical Transformer encoder layers (Section 4). Each layer applies multi-head self-attention (Eq. 19), Add & Norm, a feed-forward sublayer, and another Add & Norm. Two standard configurations were released:

Table 5: BERT model variants [10].

Variant	Layers (N)	Hidden size (d)	Heads (h)	Parameters
BERT _{BASE}	12	768	12	$\approx 110\text{M}$
BERT _{LARGE}	24	1024	16	$\approx 340\text{M}$

The output of the final encoder layer provides one contextual embedding per input token. These embeddings are consumed by the task-specific heads described in Section 6.

8. BERT Architecture

BERT is built exclusively upon the *encoder* stack of the original Transformer architecture proposed by Vaswani et al. [9]. Unlike the full encoder-decoder Transformer used for sequence-to-sequence tasks, BERT discards the decoder entirely; its objective is to produce rich contextual representations of input tokens rather than to autoregressively generate output sequences.

8.1 Transformer Encoder Stack

The model consists of L identical encoder layers stacked sequentially. Each layer $\ell \in \{1, \dots, L\}$ transforms its input representation $\mathbf{H}^{(\ell-1)} \in \mathbb{R}^{n \times d}$ (where n is the sequence length and d is the hidden dimension) into a refined output $\mathbf{H}^{(\ell)}$ through two principal sub-layers:

1. **Multi-Head Self-Attention (MHSA)**. Tokens attend to all other tokens in the sequence, producing context-enriched representations. (Described in detail in section 9.)
2. **Position-Wise Feed-Forward Network (FFN)**. A pair of linear transformations with a GELU non-linearity is applied independently to each token position.

Both sub-layers are wrapped with *residual connections* and *layer normalisation* [20], following the formulation:

$$\mathbf{H}^{(\ell)} = \text{LayerNorm}(\mathbf{z} + \text{SubLayer}(\mathbf{z})), \quad \mathbf{z} = \mathbf{H}^{(\ell-1)}. \quad (15)$$

Residual connections mitigate the vanishing-gradient problem and stabilise training in deep networks, while layer normalisation normalises activations across the hidden dimension for each token independently.

Table 6: Computation flow within a single Transformer encoder layer. MHSA denotes Multi-Head Self-Attention; FFN denotes Position-Wise Feed-Forward Network.

Sub-layer	Input	Output Dimension
MHSA	$\mathbf{H}^{(\ell-1)} \in \mathbb{R}^{n \times d}$	$\mathbb{R}^{n \times d}$
+ LayerNorm + Residual		
FFN	$\mathbf{H}^{(\ell)} \in \mathbb{R}^{n \times d}$	$\mathbb{R}^{n \times d}$
+ LayerNorm + Residual		

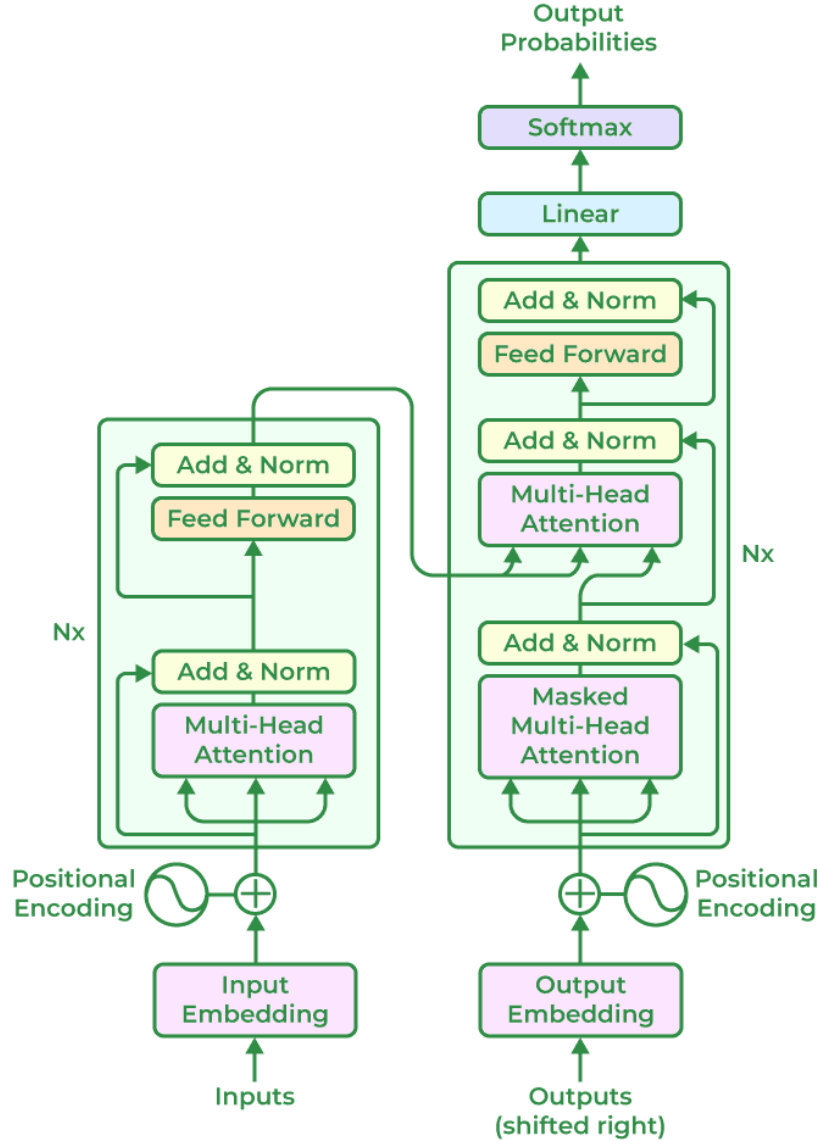


Figure 6: Stacked Transformer encoder architecture underlying BERT. Each of the L encoder layers applies Multi-Head Attention followed by a Feed-Forward Network, with residual connections and layer normalisation at each sub-layer.

8.2 Hierarchical Representation Learning

A salient property of deep Transformer encoders is their ability to construct *hierarchical* representations across layers.

- **Lower layers** ($\ell \approx 1-4$) tend to encode surface-level and syntactic information such as part-of-speech tags, morphology, and local phrase structure [21].
- **Middle layers** ($\ell \approx 5-8$) capture syntactic dependencies including subject-verb agreement and coreference.
- **Upper layers** ($\ell \approx 9-12$ for BERT-Base) encode high-level semantic information and task-relevant abstractions, making them particularly well-suited as features for downstream tasks.

This layered abstraction mirrors the hierarchical feature extraction observed in convolu-

tional neural networks applied to vision tasks, and it affords BERT the ability to generalise across a broad range of linguistic phenomena.

8.3 Model Configurations

Two standard configurations of BERT were released, differing in depth and width:

Table 7: Standard BERT model configurations. L : number of encoder layers; H : hidden dimension; A : number of attention heads; P : total parameter count.

Variant	L	H	A	P
BERT-Base	12	768	12	$\approx 110\text{M}$
BERT-Large	24	1024	16	$\approx 340\text{M}$

BERT-Large achieves higher performance on most benchmarks but demands substantially greater computational resources for both pre-training and fine-tuning. BERT-Base remains the standard choice for resource-constrained settings.

9. Self-Attention Mechanism

Self-attention is the computational core of BERT and the mechanism that enables every token to directly interact with every other token, regardless of their positional distance. Unlike recurrent architectures that process sequences step-by-step, the self-attention operation is executed in a *single parallelisable pass*, making it both expressive and computationally efficient on modern hardware.

9.1 Conceptual Overview

At an intuitive level, self-attention allows the model to ask, for each token position: “Given what I am looking for (*query*), which other tokens in the sequence (*keys*) hold the information (*values*) I should incorporate into my updated representation?” The answer is a *weighted sum* of value vectors, where the weights reflect the semantic relevance between queries and keys.

This formulation confers three key properties:

- **Global receptive field.** A token at position i can directly attend to a token at position j with $O(1)$ operations, bypassing the $O(n)$ path length required by RNNs.
- **Dynamic, content-based weighting.** Attention weights are computed on-the-fly from the current input representations rather than fixed by positional priors.
- **Parallelism.** All attention weights across all positions can be computed simultaneously, enabling efficient GPU/TPU utilisation.

9.2 Mathematical Formulation

Given an input matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$, three projection matrices $\mathbf{W}^Q, \mathbf{W}^K \in \mathbb{R}^{d \times d_k}$ and $\mathbf{W}^V \in \mathbb{R}^{d \times d_v}$ are used to produce:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}^K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}^V. \quad (16)$$

The *scaled dot-product attention* is then:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}, \quad (17)$$

where the scaling factor $1/\sqrt{d_k}$ prevents the dot products from growing large in magnitude, which would push the softmax into regions of extremely small gradients [9].

Remark 9.1. The raw score matrix $\mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{n \times n}$ contains one scalar score $e_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j$ for every ordered token pair (i, j) . Row-wise softmax converts these into a valid probability distribution; the output for position i is thus a convex combination of all value vectors, weighted by relevance to position i .

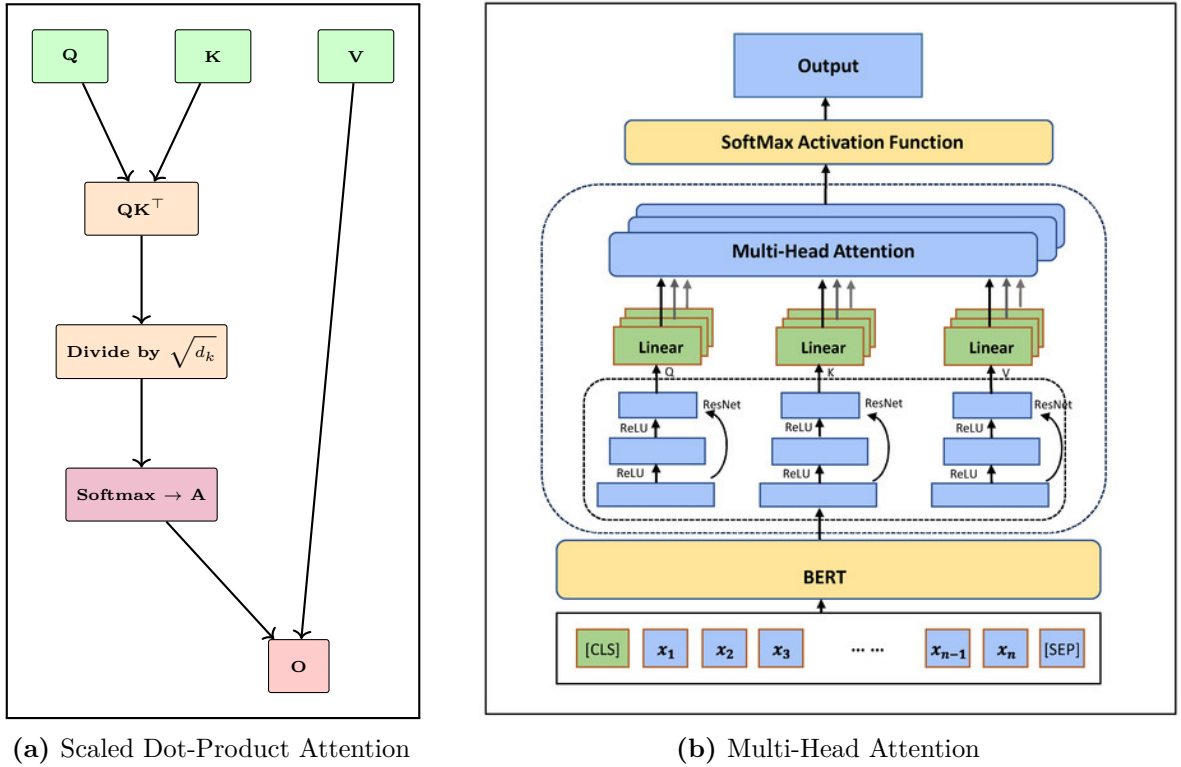


Figure 7: Attention mechanisms in BERT. (Left) Scaled dot-product attention computes relevance scores from queries and keys, applies softmax normalization to obtain attention weights $\mathbf{A}$, then aggregates values. (Right) Multi-head attention applies h parallel scaled dot-product functions in projected subspaces.

9.3 Interpretation of Query, Key, and Value Roles

- **Queries** encode what a given token is *seeking* from the rest of the sequence.
- **Keys** encode what information each token *offers* to other positions.
- **Values** carry the actual *content* to be aggregated into the output representation.

This abstraction is inspired by retrieval systems in information theory and provides a powerful inductive bias for relational reasoning over sequences.

9.4 Multi-Head Attention

Rather than computing a single attention function, BERT employs A parallel *attention heads*, each operating in a lower-dimensional subspace $d_k = d_v = d/A$:

$$\text{head}_a = \text{Attention}(\mathbf{Q}\mathbf{W}_a^Q, \mathbf{K}\mathbf{W}_a^K, \mathbf{V}\mathbf{W}_a^V), \quad a = 1, \dots, A. \quad (18)$$

Table 8: Attention head configuration for standard BERT models. Each head operates independently in a projected subspace before concatenation.

Model	d	A	d_k	d_v	Params/Head
BERT-Base	768	12	64	64	$\approx 49\text{K}$
BERT-Large	1024	16	64	64	$\approx 65\text{K}$

The outputs of all heads are concatenated and projected back to dimension d :

$$\text{MHSA}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = [\text{head}_1 \parallel \dots \parallel \text{head}_A] \mathbf{W}^O, \quad (19)$$

where $\mathbf{W}^O \in \mathbb{R}^{d \times d}$ is an output projection matrix. Multiple heads allow the model to simultaneously attend to information from different representational subspaces — e.g., one head may specialise in syntactic subject–verb relations while another focuses on coreference.

9.5 Position-Wise Feed-Forward Network

Following the multi-head attention sub-layer, each encoder layer applies an identical two-layer feed-forward network independently to each token position:

$$\text{FFN}(\mathbf{x}) = \text{GELU}(\mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2, \quad (20)$$

where $\mathbf{W}_1 \in \mathbb{R}^{d \times d_{\text{ff}}}$, $\mathbf{W}_2 \in \mathbb{R}^{d_{\text{ff}} \times d}$, and the inner dimension is $d_{\text{ff}} = 4d$ (e.g., 3072 for BERT-Base). The GELU activation function [22] is a smooth approximation to ReLU that has been found to perform better empirically in deep Transformers.

10. Input Representation

BERT does not operate on raw text but on a structured token sequence derived through a combination of three additive embedding types. This design allows a single model to handle both single-sentence and sentence-pair inputs without any architectural modification.

10.1 Tokenisation

Input text is first tokenised using *WordPiece* [23], a subword segmentation algorithm that decomposes rare or unseen words into frequent sub-units drawn from a fixed vocabulary of 30,000 tokens. For example, the word **playing** may be split into **play** and **##ing**, where the prefix **##** denotes a continuation piece. This strategy balances vocabulary coverage against embedding table size and reduces the out-of-vocabulary rate to zero.

Table 9: WordPiece tokenisation examples. Rare and out-of-vocabulary words are decomposed into frequent subword units.

Original Word	WordPiece Tokens
playing	[play, ##ing]
unbelievable	[un, ##believ, ##able]
transformer	[transform, ##er]
##ing	[##ing] (subword only)

10.2 Embedding Components

Let n denote the sequence length. Each input position $i \in \{1, \dots, n\}$ is associated with three embedding vectors, all of dimension d :

1. **Token embeddings** $\mathbf{e}_i^{\text{tok}}$: a learned lookup table that maps each WordPiece token to a dense vector in $\mathbb{R}^d$.
2. **Segment embeddings** $\mathbf{e}_i^{\text{seg}}$: one of two learned vectors ($\mathbf{E}_A$ or $\mathbf{E}_B$) indicating whether the token belongs to the first or second sentence in a sentence-pair input. For single-sentence inputs, all tokens receive $\mathbf{E}_A$.
3. **Position embeddings** $\mathbf{e}_i^{\text{pos}}$: learned vectors for each absolute position $i \in \{0, \dots, 511\}$, enabling the model to encode word order. Note that these are *learned* rather than the sinusoidal encodings used in the original Transformer.

The final input representation for position i is the element-wise sum:

$$\mathbf{h}_i^{(0)} = \mathbf{e}_i^{\text{tok}} + \mathbf{e}_i^{\text{seg}} + \mathbf{e}_i^{\text{pos}}, \quad (21)$$

and the resulting matrix $\mathbf{H}^{(0)} \in \mathbb{R}^{n \times d}$ is passed as input to the first encoder layer.

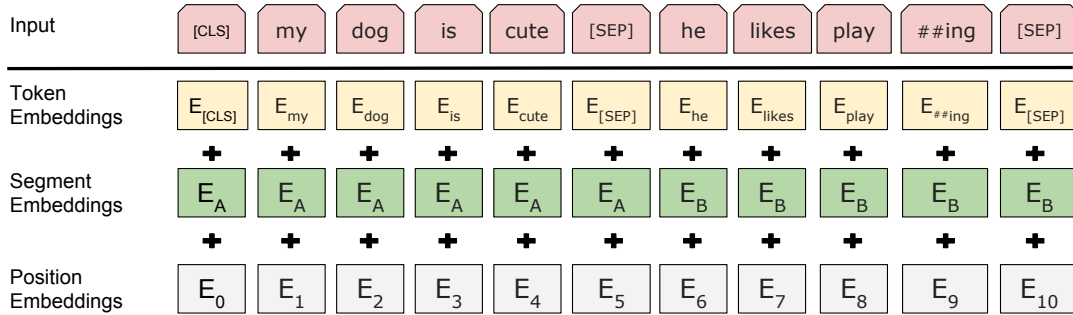


Figure 8: BERT input representation for a sentence pair. Each token embedding is summed with the corresponding segment embedding and position embedding. The special [CLS] token is prepended and [SEP] tokens delimit sentences.

10.3 Special Tokens

BERT introduces two special tokens that carry structural significance:

- [CLS] (*classification token*): prepended to every input sequence. After passing through all L encoder layers, the final hidden state corresponding to [CLS] aggregates a sentence-level (or sentence-pair-level) representation and is used as input to classification heads during fine-tuning.

- [SEP] (*separator token*): inserted at the end of each sentence in a pair to provide an explicit boundary signal. For single-sentence inputs, a single [SEP] is appended at the end.

Remark 10.1. *The maximum input length supported by BERT is 512 tokens (determined by the range of its learned position embeddings). For documents exceeding this limit, truncation or sliding-window strategies must be employed.*

11. Pre-training Strategy

BERT is pre-trained via *self-supervised learning* on large unlabelled corpora. Two complementary objectives are optimised jointly, allowing the model to acquire both token-level and sentence-level linguistic understanding.

11.1 Masked Language Modelling (MLM)

The MLM objective is the principal innovation that distinguishes BERT from left-to-right pre-training approaches.

Procedure. At each training step, 15% of the WordPiece tokens in an input sequence are selected uniformly at random. Of these selected tokens:

- 80% are replaced by the special [MASK] token;
- 10% are replaced by a randomly sampled vocabulary token; and
- 10% are left unchanged.

The model is then trained to predict the *original* token at each masked position using its final hidden state, producing the probability:

$$P(w_i \mid \mathbf{x}_{\setminus i}) = \text{softmax}(\mathbf{W}_o \mathbf{h}_i^{(L)} + \mathbf{b}_o), \quad (22)$$

where $\mathbf{h}_i^{(L)}$ is the top-layer hidden state for position i , $\mathbf{W}_o \in \mathbb{R}^{|\mathcal{V}| \times d}$ is a learned output projection, and $|\mathcal{V}|$ is the vocabulary size.

Significance. Because the hidden state for a masked token is computed from its *full bilateral context*, the model is forced to integrate information from both directions simultaneously. This is fundamentally different from autoregressive LMs, where causally masked attention prevents right-to-left information flow.

Training signal quality. The 10% random replacement and 10% unchanged strategies are crucial: they prevent the model from learning that [MASK] always indicates a position to predict, while still providing the majority of gradient signal through masked positions.

Table 10: Masked Language Modelling (MLM) token treatment strategy. For each of the 15% selected tokens, the replacement is chosen probabilistically.

Action	Proportion	Purpose
Replace with [MASK]	80%	Primary training signal
Replace with random token	10%	Encoder robustness
Keep unchanged	10%	Prevent [MASK] recognition

11.2 Next Sentence Prediction (NSP)

Many downstream tasks — notably natural language inference, question answering, and reading comprehension — require the model to reason about relationships *between* sentences. To pre-train this capability, BERT introduces an auxiliary binary classification task.

Construction. For each training example, a sentence pair (A, B) is constructed as follows:

- With probability 50%: B is the sentence that actually follows A in the source corpus (IsNext).
- With probability 50%: B is a sentence sampled uniformly at random from the corpus (NotNext).

The final hidden state of the [CLS] token is fed into a binary classifier to predict whether the pair is contiguous.

Limitations. Subsequent work [16] demonstrated that the NSP objective provides limited benefit and can even hurt performance when the training data is already richly informative. RoBERTa, a later variant, omits NSP entirely without degradation.

11.3 Pre-training Corpora

BERT is pre-trained on two large English corpora:

- **BooksCorpus** [24]: approximately 800M words of unpublished books, providing long contiguous passages essential for learning inter-sentence relations.
- **English Wikipedia**: approximately 2,500M words (text only; lists, tables, and headers excluded), providing factual and encyclopaedic knowledge.

In total, pre-training uses roughly 3.3B tokens and requires training for 1 000 000 steps with a batch size of 256 sequences, amounting to approximately 4 days on 64 Cloud TPU v3 chips for BERT-Base.

11.4 Key Observations

1. **Implicit linguistic supervision.** No hand-labelled data is used; supervision is derived entirely from the structure of natural language text.
2. **Bidirectionality.** The MLM objective produces representations that encode context from both directions, unlike GPT-style left-to-right pre-training.

3. **High transferability.** The resulting representations generalise across tasks with orthogonal linguistic requirements, requiring only a small task-specific fine-tuning head.

12. Fine-Tuning

Fine-tuning adapts the pre-trained BERT model to a specific downstream task by appending a lightweight task-specific head and training *all* parameters end-to-end on labelled task data. This stands in contrast to *feature-based* approaches (e.g., ELMo) that freeze the pre-trained encoder and use its representations as static features.

12.1 General Procedure

Fine-tuning proceeds as follows:

1. Initialise the model with the pre-trained BERT weights.
2. Construct the task-appropriate input format, inserting [CLS] and [SEP] tokens as required.
3. Append a task-specific output head (typically one or two linear layers) on top of the relevant BERT output representation.
4. Train the full model (encoder + head) jointly on the labelled training set using a small learning rate (typically $2\text{--}5 \times 10^{-5}$) for a small number of epochs (typically 3–5).

The key advantages of this approach are its simplicity and its modest data requirements: even with a few thousand labelled examples, fine-tuned BERT often surpasses task-specific architectures trained on orders of magnitude more data.

12.2 Task Formulations

12.2.1 Single-Sentence Classification

Tasks such as sentiment analysis (SST-2) and linguistic acceptability (CoLA) [12] use a single sentence as input. The final hidden state of [CLS] is passed to a linear classifier:

$$\hat{y} = \text{softmax}(\mathbf{W}_c \mathbf{h}_{[\text{CLS}]}^{(L)} + \mathbf{b}_c), \quad (23)$$

where $\mathbf{W}_c \in \mathbb{R}^{|\mathcal{Y}| \times d}$ and $\mathcal{Y}$ is the label set.

12.2.2 Sentence-Pair Classification

Tasks such as Natural Language Inference (MNLI), paraphrase detection (MRPC), and semantic similarity (STS-B) receive a sentence pair (A, B) formatted as:

$$[\text{CLS}] \ A \ [\text{SEP}] \ B \ [\text{SEP}].$$

The [CLS] representation aggregates cross-sentence information through bidirectional attention and is classified as above.

12.2.3 Question Answering (*Extractive*)

For extractive QA tasks such as SQuAD v1.1 [13], the question and passage are concatenated as the sentence pair, and the model predicts a contiguous *answer span* (s, e) within the passage. Two learned vectors $\mathbf{w}_s, \mathbf{w}_e \in \mathbb{R}^d$ compute start and end logits over all passage tokens:

$$P_s(i) = \frac{\exp(\mathbf{w}_s^\top \mathbf{h}_i^{(L)})}{\sum_j \exp(\mathbf{w}_s^\top \mathbf{h}_j^{(L)})}, \quad P_e(i) = \frac{\exp(\mathbf{w}_e^\top \mathbf{h}_i^{(L)})}{\sum_j \exp(\mathbf{w}_e^\top \mathbf{h}_j^{(L)})}. \quad (24)$$

The predicted span is $\arg \max_{s \leq e} P_s(s) \cdot P_e(e)$.

12.2.4 Named Entity Recognition (*NER*)

For sequence labelling tasks (e.g., CoNLL-2003 NER), a linear classification layer is applied to *every* token’s final hidden state independently, yielding per-token label probabilities. The [CLS] and [SEP] tokens are excluded from the loss computation.

Table 11: BERT fine-tuning results on representative NLU benchmarks. GLUE is a composite score; others report default metrics (Acc = accuracy, F1 = harmonic mean).

Task	Type	Train Size	Metric	BERT-Base
MNLI	Sentence-pair (3-way)	393K	Acc	84.6%
QQP	Paraphrase detection	364K	Acc/F1	91.3%/88.2%
MRPC	Paraphrase	3.7K	Acc/F1	89.3%/85.4%
SST-2	Sentiment (2-way)	67K	Acc	93.2%
SQuAD v1.1	Extractive QA	88K	F1/EM	88.5%/80.8%
CoNLL-2003	NER (token-level)	15K sent	F1	96.4%

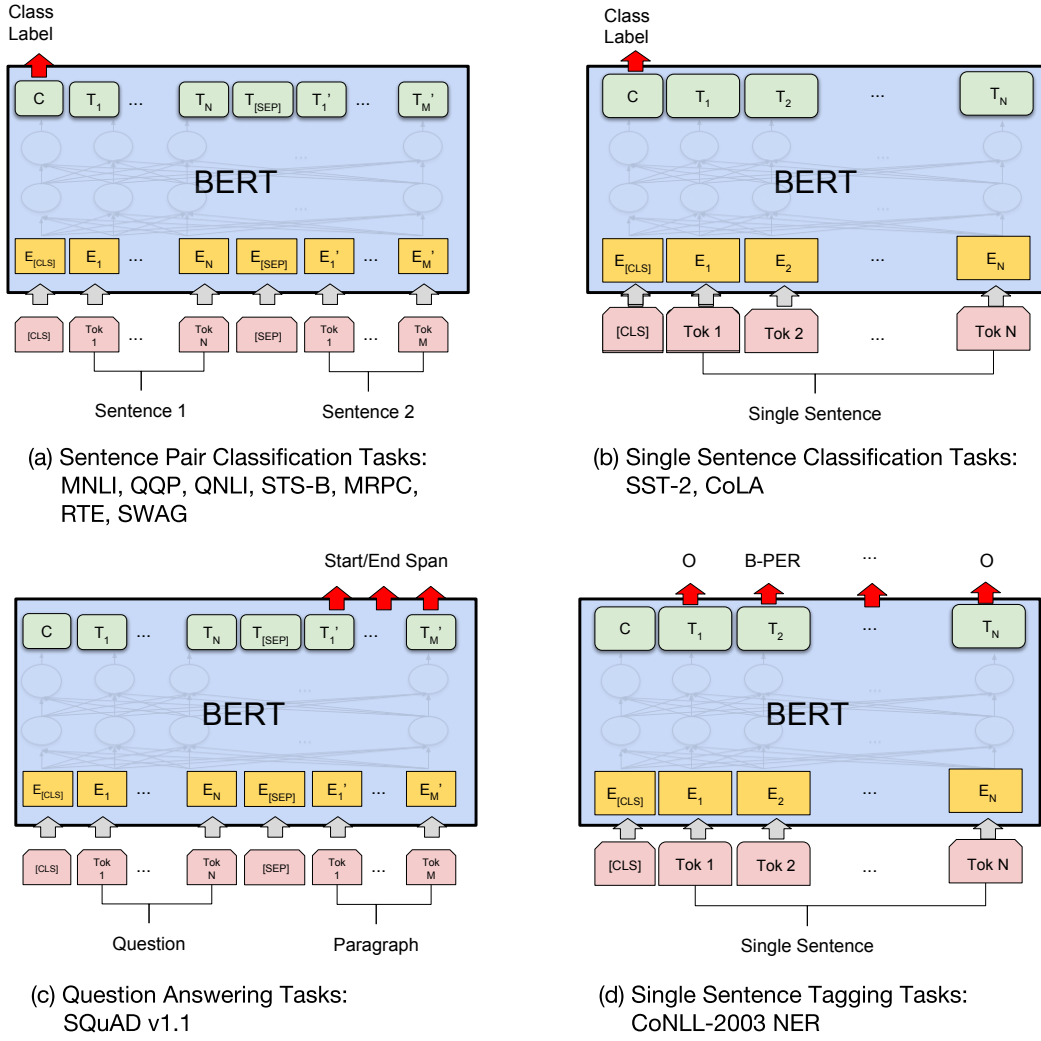


Figure 9: BERT fine-tuning architectures for four task types. (a) Sentence-pair classification; (b) single-sentence classification; (c) extractive question answering; (d) sequence labelling (NER). Only the top-layer head changes across tasks; the encoder is shared.

12.3 Practical Considerations

- **Learning rate warm-up.** A linear warm-up over the first 10% of training steps followed by linear decay is recommended to prevent catastrophic forgetting of pre-trained knowledge.
- **Batch size.** Larger batch sizes (32–128) generally improve stability on small datasets; gradient accumulation can approximate large batches on memory-constrained hardware.
- **Layer-wise learning rate decay.** Assigning lower learning rates to lower encoder layers and higher rates to the task head has been reported to improve fine-tuning stability [25].

13. Discussion and Impact

13.1 Key Contributions

BERT introduced three innovations that, taken together, established a new paradigm for NLP research and engineering:

1. **Deep bidirectional pre-training.** By employing the MLM objective, BERT learns representations that incorporate context from both directions simultaneously in every layer. This is a departure from the shallow bidirectionality of ELMo (which concatenates unidirectional LMs) and from the strict left-to-right conditioning of GPT.
2. **Unified pre-train–fine-tune framework.** A single model can be adapted to diverse tasks — classification, question answering, sequence labelling, semantic similarity — with minimal architectural change, reducing the need for highly specialised task-specific designs.
3. **State-of-the-art empirical results.** Upon publication, BERT achieved new records on 11 NLP benchmarks, including a 7.7-point improvement on the GLUE benchmark [12] and a 1.5 F1-point gain on SQuAD v1.1 [13].

13.2 Advantages

- **Strong transfer learning.** Pre-trained BERT representations encode broad linguistic knowledge that transfers well to low-resource tasks where labelled data is scarce.
- **Task flexibility.** The same encoder is applicable to sentence-level and token-level tasks without structural modification.
- **Contextual disambiguation.** Because each token’s representation is a function of all other tokens in the sequence, polysemous words are disambiguated naturally — a capability absent from static embeddings.

13.3 Limitations

Despite its impact, BERT carries several notable limitations:

- **Computational cost.** Pre-training BERT-Large requires thousands of TPU hours and is prohibitively expensive for most academic and industrial teams without specialised hardware. Inference latency is also high relative to simpler models, hampering deployment in latency-sensitive applications.
- **Fixed input length.** The 512-token limit imposed by learned position embeddings makes BERT unsuitable for long documents without chunking heuristics.
- **NSP objective limitations.** The NSP task does not require fine-grained linguistic understanding and can be learned by superficial topic-matching heuristics [16].
- **Masked token proportion.** Only 15% of tokens are masked per step, yielding a sparse training signal relative to autoregressive models that compute loss over all tokens.
- **Discrepancy between pre-training and fine-tuning.** The [MASK] token appears during pre-training but never during fine-tuning, creating a distribution mismatch that may degrade performance.

13.4 Influence and Subsequent Work

BERT’s design has spawned a rich ecosystem of variants and successors, including:

Table 12: BERT and contemporary models: a comparison of key properties. All models use the Transformer encoder architecture; differences lie in pre-training objectives, corpus size, and downstream performance.

Model	Type	Bidirectional	GLUE Score	Year
ELMo	Feature-based	Shallow	76.2	2018
BERT-Base	Pre-train/Fine-tune	Yes (MLM)	78.3	2018
GPT	Pre-train/Fine-tune	No (causal)	72.3	2018
RoBERTa	Pre-train/Fine-tune	Yes (dynamic MLM)	80.5	2019
ALBERT	Pre-train/Fine-tune	Yes (param. sharing)	75.3	2019

- **RoBERTa** [16]: removes NSP, trains with dynamic masking and larger batches, and demonstrates that pre-training for longer substantially improves performance.
- **ALBERT** [18]: introduces parameter sharing across layers and factorised embedding parameterisation to reduce model size.
- **DistilBERT** [17]: uses knowledge distillation to produce a 40%-smaller, 60%-faster model retaining 97% of BERT’s GLUE score.
- **Domain-specific BERTs**: BioBERT, SciBERT, ClinicalBERT, and others apply the same pre-training procedure to specialised corpora, demonstrating the generality of the framework.

13.5 Conclusion

BERT marks a watershed moment in the evolution of NLP. Its introduction of deep bidirectional pre-training via masked language modelling demonstrated conclusively that a single general-purpose encoder, pre-trained at scale, can outperform highly engineered task-specific architectures across the full spectrum of language understanding tasks. The architectural and training insights embedded in BERT continue to underpin state-of-the-art large language models, and its influence on the field remains profound.

14. Applications of BERT

One of BERT’s most significant contributions is the breadth and versatility of its downstream applicability. Because BERT is pre-trained once on massive corpora and subsequently *fine-tuned* with minimal task-specific supervision, a single unified backbone can be adapted to a wide range of natural language processing tasks. This section examines four representative application domains: sentiment analysis, search intent understanding, question answering, and named entity recognition.

14.1 Sentiment Analysis

Sentiment analysis—the task of classifying the emotional polarity of a piece of text—is one of the most direct demonstrations of BERT’s text-classification capability. The input is formatted as a single sequence prefixed with the [CLS] token and terminated with [SEP]; after fine-tuning, the contextual representation of [CLS] is passed through a linear classification head whose output dimension equals the number of sentiment classes.

Three representative examples illustrate the precision of BERT’s predictions. A clearly positive review such as *“I absolutely love the new product design!”* yields a probability distribution strongly concentrated on the positive class ($P_{\text{Pos}} = 0.98$, $P_{\text{Neu}} = 0.015$, $P_{\text{Neg}} = 0.005$). An ambiguous, informationally neutral query such as *“how do I place orders”* is correctly assigned to the neutral class ($P_{\text{Neu}} = 0.997$). Conversely, an implicitly negative statement such as *“this is not the answer I was looking for”* is classified as negative with high confidence ($P_{\text{Neg}} = 0.995$), demonstrating BERT’s ability to capture negation through bidirectional context. It is worth noting that BERT’s WordPiece tokenizer may segment rare words into subword units prefixed with ##, but this has no adverse effect on sentiment classification because contextual representations aggregate information across the full token sequence [10].

14.2 Search Intent Understanding — Google Search Integration

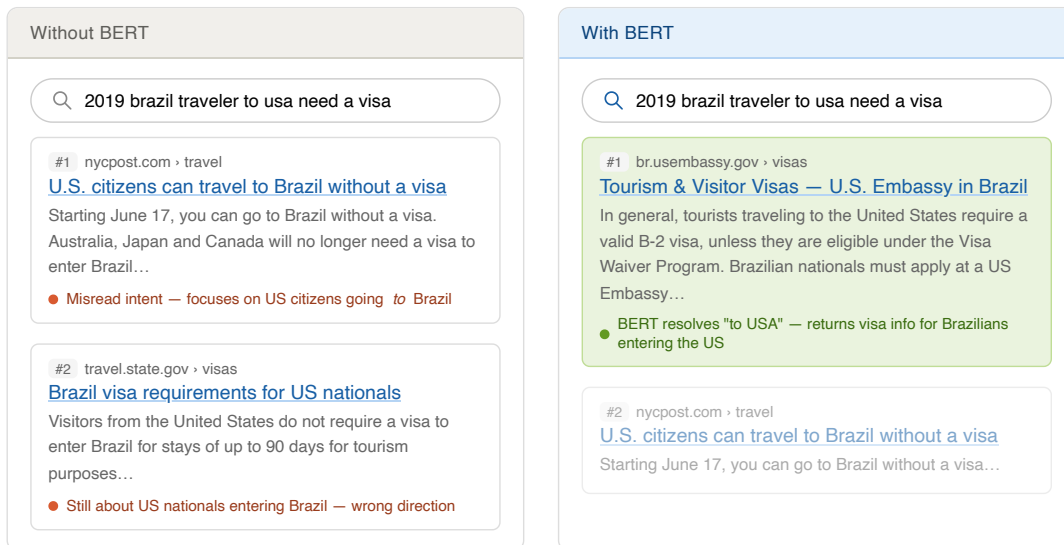


Figure 10: Impact of BERT on Search Intent Resolution. The left panel (Without BERT) shows results focusing on U.S. citizens traveling *to* Brazil, failing to capture the directional intent. The right panel (With BERT) correctly identifies the query “2019 brazil traveler to usa” as a request for U.S. entry requirements for Brazilian nationals.

In October 2019, Google deployed BERT to improve the interpretation of natural language search queries, affecting approximately one in ten English searches at launch [26]. The central challenge in web search is not merely matching keywords but understanding the *intent* embedded in a query, which often hinges on subtle linguistic cues such as prepositions or word order.

Two canonical examples underscore this point. For the query *“2019 Brazil traveler to USA need a visa”*, pre-BERT systems retrieved results focused on US citizens travelling *to*

Brazil—misreading the directionality signalled by the preposition *to*. After integrating BERT, the top result correctly pointed to US Embassy visa requirements for Brazilian nationals, because BERT conditions each token’s representation on its full bilateral context and thus correctly resolves the directional relationship.

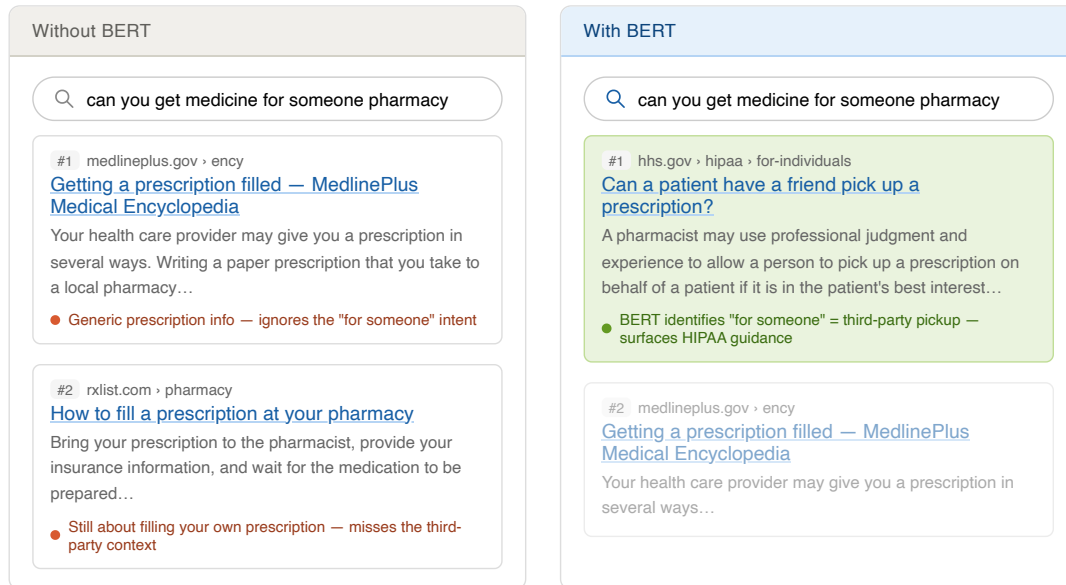


Figure 11: Close-up of BERT Contextual Understanding. Detailed view of the third-party intent resolution for pharmacy pickup queries.

A second example involves the query “*Can you get medicine for someone pharmacy*”. Earlier systems surfaced general information about filling prescriptions, overlooking the phrase “*for someone*” which signals a third-party pickup scenario. BERT’s contextualisation correctly identifies this intent, ranking results from authoritative health-policy sources that directly address whether a pharmacist may dispense medication to a proxy. These examples demonstrate that even fine-grained syntactic relationships—which static keyword matching cannot capture—are correctly resolved through BERT’s deep bidirectional representations.

14.3 Question Answering

Extractive question answering requires the model to locate a contiguous span of text within a given passage that answers a posed question. BERT addresses this as a *span extraction* task: the question and the context paragraph are concatenated as [CLS] Question [SEP] Context [SEP], and two linear layers predict the start and end token indices of the answer span [10].

Consider the following context passage: “*The BERT-Large model is a powerhouse of natural language processing. It consists of 24 transformer layers and 16 attention heads. BERT-Large features 340 million parameters, allowing it to understand complex nuances in human language.*” Given the question “*How many parameters does BERT-Large have?*”, BERT assigns the highest start-position probability to the token *340* and the highest end-position probability to the token *million*, extracting the correct answer span “*340 million*”. This behaviour arises because the cross-attention between the question tokens and the context tokens guides the model towards the semantically relevant passage fragment. On the Stanford Question Answering Dataset (SQuAD 2.0), BERT achieved state-of-the-art

performance at the time of publication, surpassing human-level accuracy on the held-out test set [27].

14.4 Named Entity Recognition

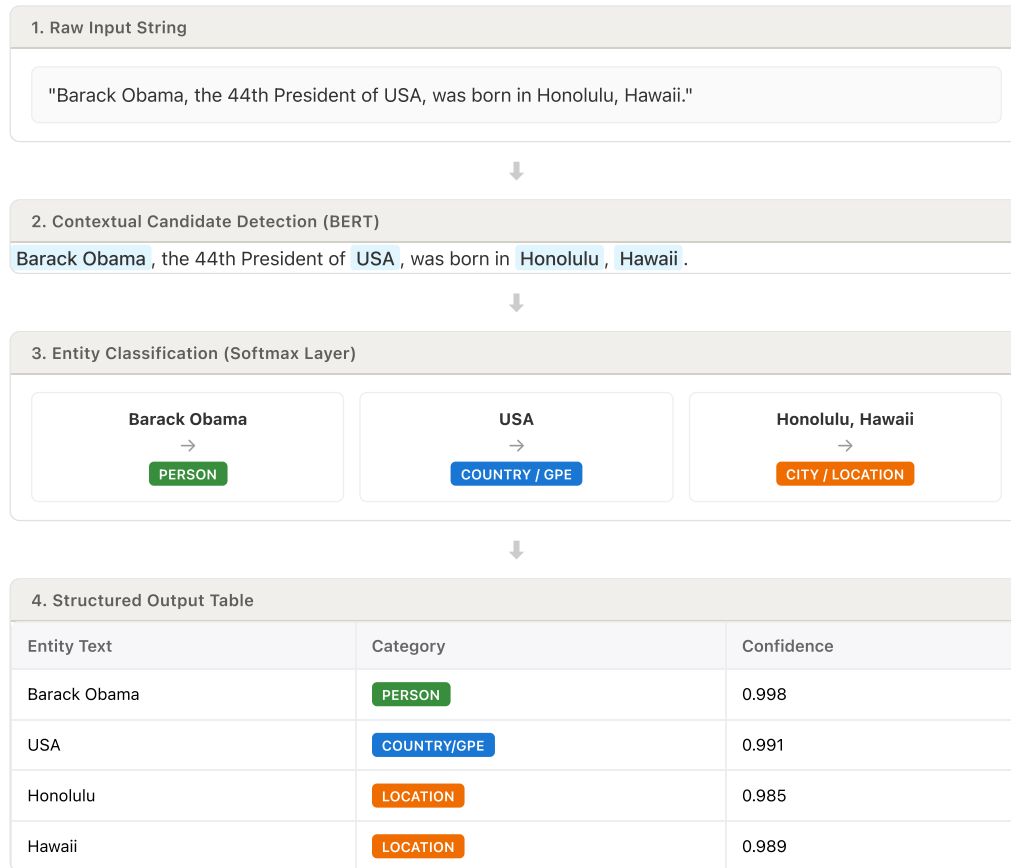


Figure 12: Four-Stage Named Entity Recognition (NER) Pipeline. The diagram illustrates the transition from (1) raw input strings to (2) contextual candidate detection via BERT, followed by (3) token-level classification into categories such as **Person**, **GPE**, and **Location**, and finally (4) the consolidation of labels into a structured output table.

Named Entity Recognition (NER) is a token-level sequence labelling task in which the model assigns a category label to each token in a sentence. Common entity categories include Person, Organisation, Location, Geopolitical Entity (GPE), and Date/Time expressions. BERT approaches NER by appending a token-level classification head atop the contextual encoder; the representation of each subword token is independently passed through a softmax layer to produce a label [10].

The pipeline can be described in four stages. First, the raw input string—for example, “*Barack Obama, the 44th President of USA, was born in Honolulu, Hawaii.*”—is tokenised and fed to the encoder. Second, the model exploits contextual signals to identify candidate entities; the phrase *Barack Obama* is highlighted as a named entity by virtue of its proper-noun characteristics and surrounding syntactic structure. Third, each detected entity is assigned its most probable label: *Barack Obama* → **Person**, *USA* → **Country/GPE**, and *Honolulu, Hawaii* → **City/Location**. Fourth, the labelled entities are collated

into a structured output table, which downstream applications can directly consume for information extraction or knowledge-graph construction.

15. Limitations of BERT

Despite its transformative impact, BERT is not without significant limitations. Understanding these shortcomings is essential both for deploying BERT responsibly and for motivating the extensive body of successor work that followed its release [28].

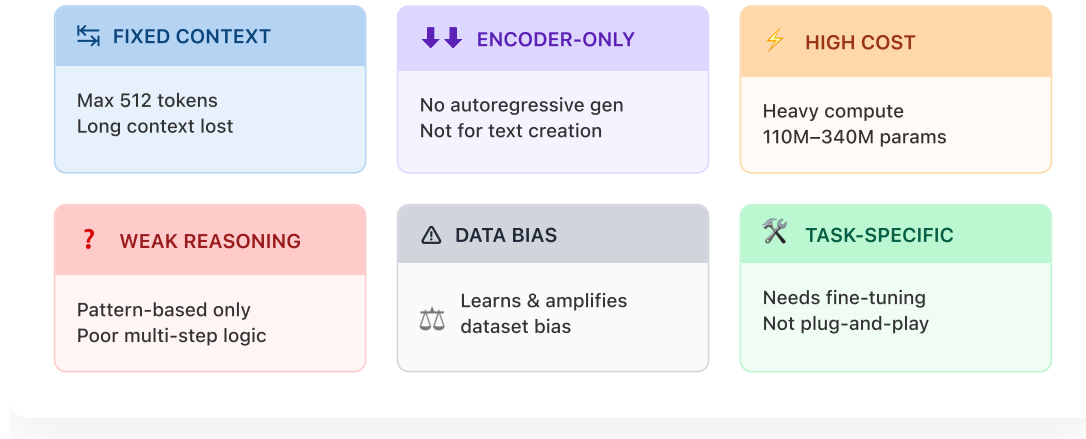


Figure 13: Limitations of BERT. Fixed context window, encoder-only design, high computational cost, weak reasoning ability, inherited data bias, and dependence on task-specific fine-tuning.

1. **Fixed context window.** BERT operates on sequences of at most 512 tokens. Documents exceeding this limit must be truncated or chunked, which breaks cross-segment dependencies and can degrade performance on long-document tasks such as legal contract analysis or scientific paper summarisation.
2. **Encoder-only architecture—no text generation.** Because BERT is a *bidirectional encoder*, it is architecturally incapable of auto-regressively generating text. Tasks requiring open-ended generation, such as machine translation, abstractive summarisation, or dialogue, require decoder-equipped architectures (e.g., GPT, T5, BART) [29].
3. **Computational cost.** BERT-Base contains 110 million parameters and BERT-Large 340 million. Both require substantial GPU memory and inference time, making real-time deployment in resource-constrained environments (e.g., on-device mobile applications) challenging without additional compression [17].
4. **Limited commonsense and logical reasoning.** BERT’s pre-training objectives—Masked Language Modelling and Next Sentence Prediction—equip it with strong distributional semantics but do not explicitly train it to perform multi-step reasoning or apply world knowledge consistently. This manifests as errors on tasks requiring numerical reasoning, causal inference, or structured commonsense knowledge [28].
5. **Inherited data bias.** BERT is pre-trained on BookCorpus and English Wikipedia. These corpora encode various societal biases pertaining to gender, race, and cultural perspective. Studies have shown that BERT’s representations reflect and can amplify

such biases, which raises ethical concerns in applications involving hiring, credit scoring, or content moderation [30].

6. **Task-specific dependency.** BERT is not a general-purpose, out-of-the-box solution. Its pre-training objective (masked language modeling) does not directly align with most downstream tasks, requiring additional fine-tuning on labeled data for each specific application. As a result, performance is highly dependent on task-specific datasets and supervision, limiting transferability and increasing deployment cost.

16. What Came After BERT

BERT’s introduction triggered an era of rapid architectural innovation aimed at addressing each of its known limitations while preserving or extending its core strengths. The resulting family of models can be broadly categorised into three groups: training-efficiency improvements, context and modelling extensions, and deployment-friendly distillations.

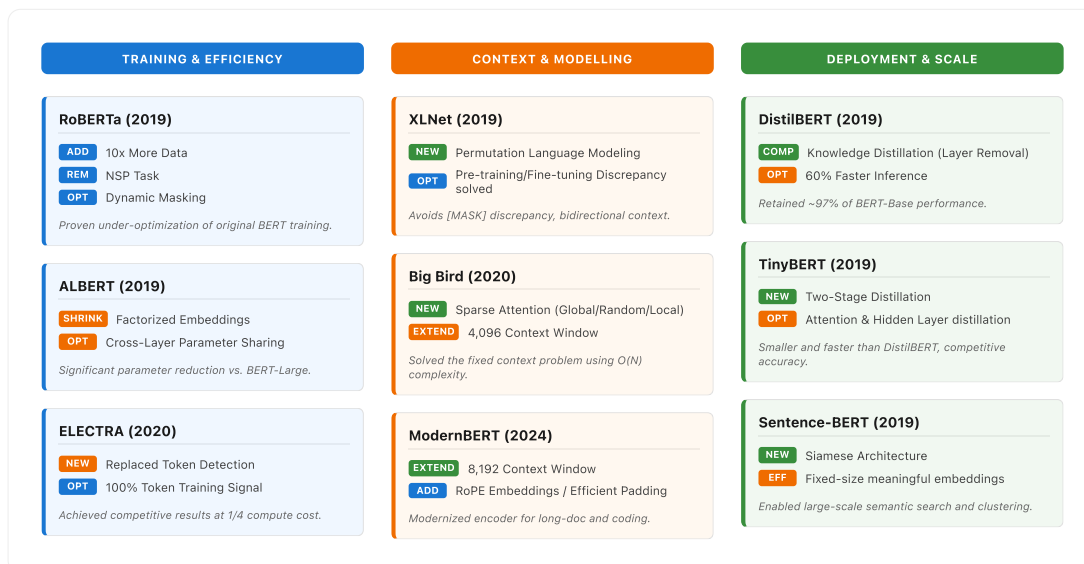


Figure 14: BERT Successors and Improvements. Overview of key BERT-based models that address limitations in pre-training, model efficiency, context length, reasoning, and task adaptability.

16.1 Training and Efficiency Improvements

RoBERTa (Robustly Optimised BERT Pre-training Approach, 2019) demonstrated that BERT’s original training was substantially under-optimised. By training on ten times more data, removing the Next Sentence Prediction objective, and adopting *dynamic masking* (so that each training pass sees a different random mask), Liu et al. achieved consistent improvements across all GLUE benchmarks without any architectural change [16].

ALBERT (A Lite BERT, 2019) introduced two parameter-reduction techniques: *factorised embedding parameterisation*, which decouples the vocabulary embedding size from the hidden layer size, and *cross-layer parameter sharing*, where all Transformer layers share identical weights. These changes reduced the parameter count by up to 89% relative to BERT-Large while maintaining competitive accuracy [18].

ELECTRA (Efficiently Learning an Encoder that Classifies Token Replacements Accurately, 2020) replaced the MLM objective with a more sample-efficient *replaced-token detection* pre-training signal: a small generator network corrupts the input and the main discriminator learns to identify which tokens were replaced. Because every token contributes a training signal (rather than only the 15% masked tokens in standard MLM), ELECTRA reached BERT’s performance with substantially less compute [31].

16.2 Context and Modelling Extensions

XLNet (2019) abandoned the masked-token paradigm in favour of *permutation language modelling*: by training the model to predict tokens in all possible orderings of the input sequence, XLNet obtains bidirectional context without the train–inference discrepancy introduced by [MASK] tokens [32].

BigBird (2020) introduced sparse attention mechanisms combining global, local, and random attention patterns to extend BERT’s context window up to 4,096 tokens while maintaining linear computational complexity. This design addresses BERT’s fixed-context limitation, making it effective for long-document understanding and retrieval tasks [33].

ModernBERT (2024) extended the context window to 8192 tokens through Rotary Position Embeddings (RoPE), incorporated efficient padding removal to avoid unnecessary computation on padding tokens, and updated the architectural components with current best practices. These changes make ModernBERT competitive on long-document retrieval and coding tasks while retaining the familiar encoder-only paradigm [34].

16.3 Deployment-Friendly Models

DistilBERT (2019) applied *knowledge distillation* to compress BERT-Base into a model with 40% fewer parameters and 60% faster inference, retaining approximately 97% of BERT’s performance on downstream benchmarks [17].

TinyBERT (2019) applied a two-stage knowledge distillation approach to compress BERT into a smaller, faster model. By distilling both attention and hidden layers from the teacher model, TinyBERT achieves competitive accuracy with reduced parameters and inference time, enabling deployment in resource-constrained environments [35].

Sentence-BERT (SBERT, 2019) fine-tuned BERT with a Siamese network architecture and a contrastive objective, producing *fixed-size sentence embeddings* that are semantically meaningful under cosine similarity. This allows large-scale semantic textual similarity, clustering, and information retrieval tasks to be performed efficiently without the expensive cross-encoder inference required by vanilla BERT [36].

17. Conclusion

BERT represents a watershed moment in the history of natural language processing. By introducing deep bidirectional pre-training via the Masked Language Model objective, it resolved the fundamental limitation of unidirectional context that constrained all prior pre-trained representations. Its unified *pre-train then fine-tune* paradigm eliminated the need to design task-specific architectures from scratch, establishing a blueprint that the entire field subsequently adopted [10].

Table 13 summarises the principal qualitative shifts that BERT introduced, contrasting the capabilities of the NLP landscape before and after its release.

Table 13: The evolution of NLP capabilities introduced by BERT.

Before BERT	After BERT
Unidirectional context (LSTMs, ELMo)	Full bidirectional context
Separate model trained per task	One backbone, many tasks
Shallow contextual signals	Deep contextual embeddings
Expensive training for each new task	Efficient fine-tuning on small data
Sequential processing (slow)	Parallel Transformer layers (fast)

The legacy of BERT extends far beyond its benchmark scores. From its release in 2018 through RoBERTa and ALBERT in 2019, ELECTRA in 2020, and ModernBERT in 2022, each successive model refined or extended BERT’s core ideas, collectively pushing the frontier of language understanding. Today, BERT and its successors underpin production systems for web search, question answering, document classification, and information extraction at global scale. More broadly, BERT established the Transformer encoder as the canonical architecture for language understanding—a position it continues to hold, and one that has influenced the design of every major language model released since [28], [30].

References

- [1] Z. S. Harris, “Distributional structure,” *WORD*, vol. 10, no. 2–3, pp. 146–162, 1954.
- [2] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, “Distributed representations of words and phrases and their compositionality,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 26, 2013.
- [3] J. Pennington, R. Socher, and C. D. Manning, “GloVe: Global vectors for word representation,” in *Proceedings of EMNLP*, Doha, Qatar, Oct. 2014, pp. 1532–1543.
- [4] P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov, “Enriching word vectors with subword information,” *Transactions of the Association for Computational Linguistics*, vol. 5, pp. 135–146, 2017.
- [5] Y. Bengio, P. Simard, and P. Frasconi, “Learning long-term dependencies with gradient descent is difficult,” *IEEE Transactions on Neural Networks*, vol. 5, no. 2, pp. 157–166, 1994.
- [6] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.
- [7] M. Schuster and K. K. Paliwal, “Bidirectional recurrent neural networks,” *IEEE Transactions on Signal Processing*, vol. 45, no. 11, pp. 2673–2681, 1997.
- [8] M. E. Peters et al., “Deep contextualized word representations,” in *Proceedings of NAACL-HLT*, New Orleans, LA, USA, Jun. 2018, pp. 2227–2237.
- [9] A. Vaswani et al., “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [10] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of NAACL-HLT*, Association for Computational Linguistics, 2019, pp. 4171–4186.
- [11] W. L. Taylor, ““cloze procedure”: A new tool for measuring readability,” *Journalism Quarterly*, vol. 30, no. 4, pp. 415–433, 1953.
- [12] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, “GLUE: A multi-task benchmark and analysis platform for natural language understanding,” in *Proceedings of EMNLP Workshop BlackboxNLP*, 2018.
- [13] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, “SQuAD: 100,000+ questions for machine comprehension of text,” in *Proceedings of EMNLP*, Austin, TX, USA, Nov. 2016, pp. 2383–2392.
- [14] P. Rajpurkar, R. Jia, and P. Liang, “Know what you don’t know: Unanswerable questions for SQuAD,” in *Proceedings of ACL*, Melbourne, Australia, Jul. 2018, pp. 784–789.
- [15] A. Williams, N. Nangia, and S. R. Bowman, “A broad-coverage challenge corpus for sentence understanding through inference,” in *Proceedings of NAACL-HLT*, New Orleans, LA, USA, Jun. 2018, pp. 1112–1122.
- [16] Y. Liu et al., “RoBERTa: A robustly optimized BERT pretraining approach,” *arXiv preprint arXiv:1907.11692*, 2019.
- [17] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter,” *arXiv preprint arXiv:1910.01108*, 2019.

- [18] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soricut, “ALBERT: A lite BERT for self-supervised learning of language representations,” in *Proceedings of ICLR*, 2020.
- [19] Y. Wu, M. Schuster, Z. Chen, et al., “Google’s neural machine translation system: Bridging the gap between human and machine translation,” *arXiv preprint arXiv:1609.08144*, 2016.
- [20] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer normalization,” in *NeurIPS Workshop on Deep Learning Symposium*, 2016.
- [21] I. Tenney, D. Das, and E. Pavlick, “BERT rediscovers the classical NLP pipeline,” in *Proceedings of ACL*, 2019, pp. 4593–4601.
- [22] D. Hendrycks and K. Gimpel, “Gaussian error linear units (GELUs),” *arXiv preprint arXiv:1606.08415*, 2016.
- [23] Y. Wu et al., “Google’s neural machine translation system: Bridging the gap between human and machine translation,” *arXiv preprint arXiv:1609.08144*, 2016.
- [24] Y. Zhu et al., “Aligning books and movies: Towards story-like visual explanations by watching movies and reading books,” in *Proceedings of ICCV*, 2015, pp. 19–27.
- [25] C. Sun, X. Qiu, Y. Xu, and X. Huang, “How to fine-tune BERT for text classification?” In *Chinese National Conference on Chinese Computational Linguistics*, 2019, pp. 194–206.
- [26] P. Nayak, *Understanding searches better than ever before*, Google Blog, Oct. 2019. [Online]. Available: <https://blog.google/products/search/search-language-understanding-bert/>
- [27] P. Rajpurkar, R. Jia, and P. Liang, “Know what you don’t know: Unanswerable questions for SQuAD,” in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, Melbourne, Australia: Association for Computational Linguistics, 2018, pp. 784–789. DOI: [10.18653/v1/P18-2124](https://doi.org/10.18653/v1/P18-2124)
- [28] A. Rogers, O. Kovaleva, and A. Rumshisky, “A primer in BERTology: What we know about how BERT works,” *Transactions of the Association for Computational Linguistics*, vol. 8, pp. 842–866, 2020. DOI: [10.1162/tac1_a_00349](https://doi.org/10.1162/tac1_a_00349)
- [29] M. Lewis et al., “BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension,” in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, Online: Association for Computational Linguistics, 2020, pp. 7871–7880. DOI: [10.18653/v1/2020.acl-main.703](https://doi.org/10.18653/v1/2020.acl-main.703)
- [30] E. M. Bender, T. Gebru, A. McMillan-Major, and S. Shmitchell, “On the dangers of stochastic parrots: Can language models be too big?” In *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, Virtual Event, Canada: ACM, 2021, pp. 610–623. DOI: [10.1145/3442188.3445922](https://doi.org/10.1145/3442188.3445922)
- [31] K. Clark, M.-T. Luong, Q. V. Le, and C. D. Manning, “ELECTRA: Pre-training text encoders as discriminators rather than generators,” in *Proceedings of the 8th International Conference on Learning Representations (ICLR)*, 2020. [Online]. Available: <https://openreview.net/forum?id=r1xMH1BtvB>

- [32] Z. Yang, Z. Dai, Y. Yang, J. Carbonell, R. Salakhutdinov, and Q. V. Le, “XLNet: Generalized autoregressive pretraining for language understanding,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, Curran Associates, Inc., 2019. [Online]. Available: <https://arxiv.org/abs/1906.08237>
- [33] M. Zaheer et al., “Big bird: Transformers for longer sequences,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.
- [34] B. Warner et al., “Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference,” *arXiv preprint*, vol. arXiv:2412.13663, 2024. [Online]. Available: <https://arxiv.org/abs/2412.13663>
- [35] X. Jiao et al., “Tinybert: Distilling bert for natural language understanding,” in *arXiv preprint arXiv:1909.10351*, 2019.
- [36] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using siamese BERT-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Hong Kong, China: Association for Computational Linguistics, 2019, pp. 3982–3992. DOI: [10.18653/v1/D19-1410](https://doi.org/10.18653/v1/D19-1410)